

# הגשת פרוייקט הקורס 2026

כל הזכויות שמורות ליובל עוזרי 2026

# פרויקט גמר – הנחיות הגשה ודרישות

---

אין הגשות לאחר מועד ההגשה האחרון, גם לא דרך GitHub.  
במקרים מיוחדים בלבד יש לפנות למרצה לפחות יומיים לפני מועד ההגשה.

הציון בפרויקט חייב להיות לפחות 60. ציון נמוך מ־60 בפרויקט משמעו אי־מעבר בקורס.

אין להגיש פרויקט שרץ רק על `localhost`.  
הפרויקט חייב להיות זמין לבדיקה דרך קישור פעיל.

# פרויקט גמר – הנחיות הגשה ודרישות

ב-Moodle יש להגיש:

- קובץ ZIP של הפרויקט
- לינק לדף הבית של הפרויקט
- לינק ל-Figma של הפרויקט
- לינק לריפוזיטורי GitHub – צד שרת
- לינק לריפוזיטורי GitHub – צד לקוח
- לינק ל-Postman API Collection
- קובץ env. (בתוך הפרויקט עצמו, לא בנפרד)

במסמך פרטי הבדיקה יש לציין גם:

- באיזה API חיצוני השתמשתם
- באיזו ספריית JavaScript השתמשתם
- האם יש דברים מיוחדים שהבודקים צריכים לדעת לפני בדיקת הפרויקט

# עבודה עם Git ו-GitHub

חובה לעבוד עם Git ו-GitHub לאורך כל הפרויקט.

ייבדקו הקומיטים של חברי הצוות.

הקומיטים צריכים להיות משמעותיים ולהראות תהליך עבודה אמיתי לאורך זמן.  
לא יתקבל מצב שבו כל הפרויקט מועלה בקומיט אחד בסוף.

אין להעלות סיסמאות, קבצי `env`, או פרטי התחברות סודיים ל-GitHub.  
אם יש צורך בסיסמאות לבדיקה, יש לצרף אותן במסמך נפרד ב-Moodle בלבד.  
אין להעלות את תיקיית `node_modules`.

# דרישות כלליות

הפרויקט צריך לממש את העיצוב שנבנה ב-Figma בצורה מדויקת ככל האפשר.

דפים שלא עוצבו ב-Figma צריכים לשמור על אותו קו עיצובי של שאר הפרויקט.

מידע דינמי בפרויקט לא יהיה כתוב בצורה סטטית בתוך הקוד.

לדוגמה: משתמשים, מוצרים, פוסטים, הזמנות, תגובות, פריטים וכדומה צריכים להגיע מה-Database.

# צד השרת

---

צד השרת ייכתב ב-Node.js עם Express.

מבנה הפרויקט צריך להיות מסודר ולכלול חלוקה ל-routes ו-controllers, כפי שלמדנו בביתה.

ה-API צריך להיות בנוי לפי עקרונות RESTful API.

חובה לממש CRUD מלא:

- Create – הוספה
- Read – שליפה והצגה
- Update – עדכון
- Delete – מחיקה

# צד השרת

בנוסף, צריכות להיות לפחות שתי שאילתות מורכבות יותר, בהתאם לאופי הפרויקט.

לדוגמה:

- חיפוש
- סינון
- מיון
- שליפה לפי קטגוריה
- שליפה לפי משתמש
- שליפה עם קשר בין טבלאות / collections
- סטטיסטיקה או חישוב כלשהו מתוך הנתונים

השרת צריך להחזיר תשובות ברורות גם במצב הצלחה וגם במצב שגיאה.

# Postman API

יש לפרסם Postman Collection מסודר שכולל את כל ה-routes של השרת.

לכל route צריך להיות:

- הסבר קצר מה הנתיב עושה
- דוגמה לבקשה תקינה
- דוגמה לתשובה תקינה
- דוגמה למצב שגיאה, אם רלוונטי

בנוסף, יש לשמור ב-Postman דוגמאות של תשובות מהבדיקות, כך שהבודקים יוכלו לראות בקלות מה אמורה להיות התוצאה של כל בקשה.

-



# חיצוני API

בפרויקט צריכה להיות קריאה ל-API חיצוני ציבורי שקשור לנושא הפרויקט.  
ה-API החיצוני צריך להיות חלק אמיתי מהמערכת, ולא רק קריאה אקראית שלא משפיעה על האתר.

דוגמאות:

- פרויקט מזג אוויר / טיולים / אירועים – API של מזג אוויר להצגת תחזית לפי מיקום
- פרויקט אוכל / מתכונים / מסעדות – API של מתכונים או מידע על מאכלים
- פרויקט ספרים / ספרייה / קריאה – API לחיפוש ספרים לפי שם, מחבר או קטגוריה

דוגמאות ל-APIs שאפשר לבדוק:

- OpenWeather – מזג אוויר
- TheMealDB – מתכונים ואוכל
- Google Books API / Open Library – ספרים

לכל נושא כמעט יש API, אין סיכוי שלא תמצאו משהו שמתאים לפרויקט שלכם

# צד לקוח

צד הלקוח ייכתב בעזרת:

- HTML
- CSS
- JavaScript

יש לשמור על מבנה תיקיות מסודר:

- index.html
- /js
- /style
- /images

אין להשתמש ב-`confirm`, `alert`, או `prompt`. במקום זאת יש להציג הודעות למשתמש בתוך האתר, למשל הודעת הצלחה, שגיאה, `popup` או `modal` מעוצב.

אין להשתמש ב-`important` ללא סיבה מוצדקת. ברוב המקרים שימוש ב-`important` מעיד על CSS לא מסודר.

אין לכתוב עיצוב ישירות בתוך HTML בעזרת `style=""`, אלא להשתמש בקבצי CSS וב-`classes`.

# חויית משתמש

כל מה שמופיע באתר צריך לעבוד.

אין להשאיר באתר כפתורים, קישורים, טפסים, אייקונים או פעולות שלא ממומשים בפועל.

אם יש אלמנט בעיצוב שלא מתוכנן לעבוד בפרויקט, יש להסיר אותו מהאתר ולעדכן את ה-Figma בהתאם.

המשתמש צריך להבין מה קורה באתר.

בפעולות מול השרת יש להציג:

- מצב טעינה
- הודעת הצלחה
- הודעת שגיאה
- הודעה כאשר אין נתונים להצגה

האתר צריך לעבוד ללא שגיאות ב-Console בזמן שימוש רגיל.

# הצגת הפרויקט

כל קבוצה תציג את הפרויקט בכיתה.

משך ההצגה: עד 7 דקות.

בהצגה יש להראות:

- את הרעיון הכללי של הפרויקט
- שני flows מרכזיים של משתמש
- במידת הצורך, flows של סוגי משתמשים שונים
- פעולות מרכזיות מול השרת וה-Database

ההצגה היא חובה.

# חלוקת הניקוד

- 10 נק' - עיצוב לפי הפיגמה
- 10 נק' - חלוקה נכונה לאיזורים בעמוד, מבנה HTML תקין והגיוני
- 10 נק' - RESTFUL API תקני, מפובלש בפוסטמן לפי הדרישות
- 10 נק' - מבנה דטבייס תקין ויעיל, שאילתות נכונות, בדיקת התשובות החוזרות
- 10 נק' - ניהול משתמשים
- 10 נק' - פונקציונליות נכונה ויעילות
- 10 נק' - clean code: קוד מסודר ונקי, שמות משמעותיים, ללא קוד מיותר, שימוש ב-ES6
- 10 נק' - ניהול גירסאות: קומיטים משמעותיים מתחילת הפרוייקט ובמהלך העבודה
- 5 נק' - API חיצוני
- 5 נק' - הטמעת ספריית JS
- 5 נק' - ניהול שגיאות
- 5 נק' - מבנה תיקיות נכון